

Reviewer Pack — Sub-Sqrt-Log Concentration of Tropical Doubling Discrepancy ...

Entry 34f4f9958da2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 09:50:13 UTC

Sub-Sqrt-Log Concentration of Tropical Doubling Discrepancy at $\beta=5$

Entry ID: 34f4f9958da2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 09:50:13 UTC

1. Conjecture

Field A (mathematical branch): Discrepancy theory (sub-Gaussian concentration of L^∞ Fourier discrepancy under Maslov dequantization) **Field B** (complexity object): Tropical Fourier discrepancy $\Delta(f, \beta, n) := |\text{MFC}(g) - 2 \cdot \text{MFC}(f)|$ for g = tropical self-convolution of f , viewed as a complexity-theoretic proxy for min-plus communication separation

Statement:

Let $f: \mathbb{Z}_n \rightarrow \mathbb{R}$ have i.i.d. coefficients $\sim N(0,1)$ and let $g[k] := \min_{\{i+j \equiv k \pmod{n}\}} (f[i] + f[j])$ be the tropical self-convolution. Define the Maslov-dequantized TFT at $\beta=5$ by $\text{TFT}_\beta(h)[k] := -(1/\beta) \log |\sum_{j=0}^{n-1} \exp(-\beta h[j]) \cdot \exp(-2\pi i j k / n)|$, $\text{MFC}(h) := \min_{\{k \neq 0\}} |\text{TFT}_\beta(h)[k]|$, and $\Delta(f, 5, n) := |\text{MFC}(g) - 2 \cdot \text{MFC}(f)|$. Conjecture (one-step refinement of SC1 from $O(\log n)$ to $O(\sqrt{\log n})$): there exists an absolute constant $C \leq 4$ with $E[\Delta(f, 5, n)] \leq C \cdot \sqrt{\log n}$ for every $n \in \{8, 12, 16, 20, 24, 28, 32, 36, 40\}$, and equivalently the bootstrapped 95% CI for the log-log regression slope α of $E[\Delta]$ against $\log(\log n)$ is contained in $[0, 0.6]$; a single n whose 30-seed empirical mean exceeds $4 \cdot \sqrt{\log n}$, or whose bootstrap 95% CI for α exceeds 0.6, falsifies the claim.

Rationale (proposer's reasoning):

SC1 only asserts slope $\in (0, 1)$. The Maslov-dequantized TFT at finite β is a log-sum-exp soft-max whose fluctuations under Gaussian f are sub-Gaussian by Hanson-Wright-type arguments, so the L^∞ -style discrepancy Δ should inherit the classic $\sqrt{\log n}$ maximum-of- n -sub-Gaussians scaling rather than the naive $\log n$ union bound. Pinning the exponent below 0.6 tests whether the $2 \cdot \text{MFC}(f)$ and $\text{MFC}(g)$ terms exhibit the cancellation predicted by this dequantization heuristic — exposing structure invisible at $\beta = \infty$.

Taxonomy category: tropical_discrepancy_finite_beta (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4d87eeab585818b5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{8, 12, 16, 20, 24, 28, 32, 36, 40\}$ with 30 fixed seeds each, the conjecture is SUPPORTED iff (i) the 30-seed empirical mean $\mu_n = E[\Delta(f, 5, n)]$ satisfies $\mu_n \leq 4 \cdot \sqrt{\log n}$ for every n , AND (ii) the 1000-resample seed-level bootstrap 95% CI for the log-log slope α of μ_n vs $\log(\log n)$ lies entirely within $[0, 0.6]$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.99	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -tropical convolution Maslov dequantization Fourier discrepancy concentration-min-plus convolution sub-Gaussian concentration log-sum-exp discrepancy-tropical self-convolution Fourier transform communication complexity separation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_sample(mu=0, sigma=1):
    u1 = random.random()
    u2 = random.random()
    z0 = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
    return mu + sigma * z0

def min_plus_convolution(f, g, n):
    result = [float('inf')] * n
    for i in range(n):
        for j in range(n):
            if (i + j) % n == 0:
                result[i] = min(result[i], f[i] + g[j])
    return result

def maslov_dequantized_tft(h, beta, n):
    h_min = min(h)
    h_shifted = [x - h_min for x in h]
    sum_exp_h = sum(math.exp(-beta * x) for x in h_shifted)
    tft_values = [-1 / beta * math.log(abs(sum_exp_h)) for _ in range(n)]
    return tft_values
```

```

def mfc(h, n):
    return min(abs(maslov_dequantized_tft(h, 5, n)) for k in range(1, n))

def delta(f, g, n):
    return abs(mfc(g, n) - 2 * mfc(f, n))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 12, 16, 20, 24, 28, 32, 36, 40]
    results = []

    for n in n_values:
        f = [gaussian_sample() for _ in range(n)]
        g = min_plus_convolution(f, f, n)

        delta_value = delta(f, g, n)
        results.append(delta_value)

    mean_delta = sum(results) / len(results)
    std_dev_delta = math.sqrt(sum((x - mean_delta) ** 2 for x in results) / len(results))

    return {
        "metric_name": "Delta",
        "metric_value": mean_delta,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": all(x <= 4 * math.sqrt(math.log(n)) for n, x in zip(n_values, results)),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    all_results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        all_results.append(trial_result)

    mean_delta = sum(x["metric_value"] for x in all_results) / len(all_results)
    std_dev_delta = math.sqrt(sum((x["metric_value"] - mean_delta) ** 2 for x in all_results) / len(all_results))
    support_fraction = sum(1 for x in all_results if x["conjecture_holds"]) / len(all_results)

    if all(x["conjecture_holds"] for x in all_results):
        print(f"RESULT: SUPPORTED mean={mean_delta} std={std_dev_delta} support_fraction={support_fraction}")
    elif any(not x["conjecture_holds"] for x in all_results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"not_enough_support\" first_failing_seed=<s>")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_data n_tested={len(all_results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11682fd7.py", line 75, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11682fd7.py", line 54, in run_trial
    delta_value = delta(f, g, n)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11682fd7.py", line 43, in delta
    return abs(mfc(g, n) - 2 * mfc(f, n))
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11682fd7.py", line 40, in mfc
    return min(abs(maslov_dequantized_tft(h, 5, n)) for k in range(1, n))
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11682fd7.py", line 40, in <genexpr>
    return min(abs(maslov_dequantized_tft(h, 5, n)) for k in range(1, n))
           ~~~~~
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` before producing any data, as the `mfc` function returned a list instead of a scalar to `abs()`. No empirical means or bootstrap CI were computed, so the pre-registered support condition cannot be evaluated. | next: Fix the `mfc` function to correctly index `TFT_β(h)[k]` inside the generator (currently it ignores `k` and passes the full array to `abs`), then rerun the 30-seed sweep over $n \in \{8, \dots, 40\}$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	54376
2	preregistration	claude_max	opus	0	0	13360
3	novelty	claude_max	opus	0	0	3699
4	novelty	claude_max	opus	0	0	7607
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18064
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10445
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9420
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11502
9	judge	claude_max	opus	0	0	4452

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132924 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/34f4f9958da2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/34f4f9958da2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/34f4f9958da2.tar.gz (if generated)